深度学习
第 2 章　神经网络基础

——从感知机到多层网络，把神经网络的”骨架”真正搞清楚

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

神经网络看起来很神秘，但拆开来看，不过是一堆”加权求和 + 激活函数”的重复叠加。本章 6 道题从感知机出发，一步步搭出完整的多层神经网络，并实现手写数字识别：

- **Q2-01（辨析填空）**：感知机 vs 神经网络——感知机是什么？它和现代神经网络的本质区别在哪？激活函数为什么是从感知机到神经网络的关键一步？
- **Q2-02（计算题）**：激活函数数值计算——给定输入，分别计算 Sigmoid、Tanh、ReLU、Softmax 的输出，感受各函数的”数值行为”。
- **Q2-03（对比分析）**：六种激活函数全面对比——从公式、值域、梯度特性、适用场景全方位梳理，建立激活函数选择的系统认知。
- **Q2-04（计算题）**：三层神经网络前向传播手动推导——给定权重矩阵和输入，手算各层的加权求和与激活输出，追踪数据流动的完整路径。
- **Q2-05（代码阅读）**：神经网络代码追踪——阅读一段三层神经网络的 NumPy 实现，逐行分析形状变化，补全关键缺失步骤。
- **Q2-06（综合应用）**：手写数字识别全流程——MNIST 数据集加载、前向推理、Softmax 输出解读，理解从像素到预测类别的完整过程。

1　第 2 章　神经网络基础

第 1 章我们认识了感知机——一个”加权求和，超过阈值就输出 1，否则输出 0”的简单模型。它能解决线性可分问题，但一遇到 XOR 这类非线性问题就彻底无能为力。

神经网络解决了这个问题，靠的就是两个关键升级：

- **激活函数**：用平滑的非线性函数（Sigmoid、ReLU……）替换硬性的阶跃函数，让网络有能力拟合任意复杂的非线性关系；
- **多层叠加**：一层”加权求和 + 激活”的输出作为下一层的输入，层层堆叠，表达能力随层数指数级增长。

理论上，一个足够宽的两层神经网络可以逼近任意连续函数（万能近似定理）——这就是神经网络强大的根本原因。

先建立一个总感觉：神经网络像什么？

把神经网络比作一条有多道工序的加工流水线。

- **每道工序（每一层）** = 接收上一道工序的半成品（上一层的输出），做一次线性变换（加权求和），再做一次非线性处理（激活函数），把结果传给下一道工序；
- **激活函数** = 工序中的”成型模具”：没有它，每道工序都只是线性拉伸，叠多少道都等价于一道——有了它，每层都能引入新的”形状变化”，表达能力才真正随层数增长；
- **权重** = 每道工序的操作参数：训练就是不断调整这些参数，让流水线最终输出最接近期望的产品。

Q2-01 感知机 vs 神经网络：从阶跃到平滑

题目描述：

文档第 2.1 节回顾了感知机，并指出神经网络在此基础上的关键演进。理解这个演进，才能真正明白激活函数”为什么必须存在”。

问题：

(1) 填写下表，对比感知机与神经网络的核心差异：

	感知机	**神经网络（多层）**
激活函数	阶跃函数（输出 0 或 1）	_____
激活函数是否可微	_____	几乎处处可微（支持梯度下降）
能否解决非线性问题	单层不能，多层_____	_____
参数学习方式	感知机学习规则（硬更新）	_____
典型应用	_____	图像分类、NLP、语音识别等

(2) **激活函数为什么不能用阶跃函数（在神经网络中）？**从可微性和梯度下降的角度解释。

(3) **多层线性网络的局限性**：假设一个三层网络，每层都只做线性变换（无激活函数）：

$$y = W_3(W_2(W_1x))$$

证明这等价于单层线性变换 $y = Wx$，并说明这意味着什么。

(4) **偏置项 b 的几何含义**：在 $z = Wx + b$ 中，偏置 b 的作用是什么？如果去掉偏置，网络的表达能力会受到什么限制？（提示：从决策边界是否过原点的角度思考）

来源溯源：[文档第 2 章 2.1 神经网络的构成]

小白必读：感知机和神经网络，差在哪一步？

- **感知机的激活函数**：阶跃函数——输入大于阈值输出 1，否则输出 0。这个函数在阈值处不可微（导数无意义），其余地方导数为 0，**梯度处处为零或无意义，无法用梯度下降学习**。感知机只能用专门的”感知机学习规则”，且只适用于线性可分问题。
- **神经网络的激活函数**：Sigmoid、ReLU 等平滑（或分段线性）函数，在绝大多数点可微，梯度有意义，反向传播能正常传递信号。
- **核心结论**：从感知机到神经网络，**最关键的一步就是把阶跃函数换成平滑的非线性激活函数**。这一步让梯度下降成为可能，整个深度学习的大厦才能建立起来。

参考答案与详细解析

(1) 对比表填写：

	感知机	**神经网络**
激活函数	阶跃函数（0 或 1）	Sigmoid、ReLU 等平滑非线性函数
是否可微	不可微（阈值处）/导数为 0	几乎处处可微
非线性问题	单层不能，多层理论上能（但无法有效学习）	多层可以，配合梯度下降高效学习
参数学习	感知机学习规则（错误驱动）	反向传播 + 梯度下降
典型应用	简单线性分类	图像、语音、NLP 等复杂任务

(2) 阶跃函数不能用于神经网络的原因：

梯度下降需要计算损失对参数的偏导数，并将梯度通过激活函数反向传播。阶跃函数在阈值处不可微，在其余处导数为 0——无论上游传来多大的梯度，经过阶跃函数后都变为 0，底层参数永远得不到更新信号，**反向传播完全失效**。

(3) 多层线性网络等价于单层：

由矩阵乘法的结合律：

$$y = W_3(W_2(W_1x)) = (W_3W_2W_1)x = Wx, \quad W = W_3W_2W_1$$

三个矩阵相乘仍是一个矩阵，整个”三层网络”等价于一个单层线性变换。无论叠多少层，只要没有非线性激活函数，**表达能力永远只有一个线性函数**，无法拟合任何非线性关系——增加层数毫无意义。

(4) 偏置 b 的几何含义：

在二维情形，$z = w_1x_1 + w_2x_2 + b = 0$ 是决策边界。无偏置（$b = 0$）时，决策边界**必须过原点**，极大限制了网络能拟合的分类边界的位置。加入偏置后，决策边界可以平移到任意位置，网络可以拟合任意位置的分类面，表达能力显著提升。偏置可以理解为”激活函数的偏移量”——控制神经元在什么水平的输入下才会被激活。

Q2-02 激活函数数值计算

题目描述：

文档第 2.2 节详细介绍了各种激活函数。把激活函数的公式”算出来”，比死记公式理解深得多——你会直接感受到每种函数的”数值行为”。

对输入 $x = 2.0$，分别计算各激活函数的输出（结果保留四位小数）：

对输入向量 $\mathbf{z} = [3.0,\ 1.0,\ 0.5,\ -1.0]$（用于 Softmax）：

问题：

(1) **Sigmoid 函数**：$\sigma(x) = \dfrac{1}{1+e^{-x}}$，计算 $\sigma(2.0)$。

(2) **Tanh 函数**：$\tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$，计算 $\tanh(2.0)$（可使用 $e^2 \approx 7.389$）。

(3) **ReLU 函数**：$\mathrm{ReLU}(x) = \max(0, x)$，计算 $\mathrm{ReLU}(2.0)$ 和 $\mathrm{ReLU}(-2.0)$。

(4) **Softmax 函数**：$p_k = \dfrac{e^{z_k}}{\sum_j e^{z_j}}$，对输入 $\mathbf{z} = [3.0,\ 1.0,\ 0.5,\ -1.0]$ 计算 Softmax 输出（结果保留三位小数），并验证各概率之和为 1。

(5) **Softmax 的数值稳定性技巧**：直接计算 $e^{3.0}, \ldots, e^{-1.0}$ 没问题，但若 $\mathbf{z} = [1000,\ 999,\ 998]$，直接算 e^{1000} 会数值溢出。标准的解决方案是：

$$p_k = \frac{e^{z_k - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}$$

证明这个变换不改变 Softmax 的结果，并对 $\mathbf{z} = [1000,\ 999,\ 998]$ 用此公式计算 Softmax（结果保留三位小数）。

(6) 观察规律：Sigmoid 和 Tanh 的输出值域分别是什么？ReLU 的输出值域呢？Softmax 的输出有什么约束？

来源溯源：[文档第 2 章 2.2 激活函数]

小白必读：Softmax 为什么总用在最后一层？

- **Softmax 把任意实数变成概率分布**：无论输入（logits）是正是负、多大多小，Softmax 的输出总是（1）每个值在 $(0, 1)$ 之间，（2）所有值加起来等于 1——这正是概率分布的定义。
- **为什么只在输出层用**：Softmax 会把每个神经元的输出和其他神经元**耦合**在一起（分母是所有输出的指数和），一个值变化会影响所有输出的值。在隐藏层使用 Softmax 会导致计算复杂且梯度难以传播，不如 ReLU 简洁高效。
- **Softmax 配合交叉熵**：输出层 Softmax + 交叉熵损失是多分类任务的”黄金组合”，梯度简洁（$y_k - t_k$），数值稳定（合并计算）。

参考答案与详细解析

(1) Sigmoid（$e^{-2} \approx 0.1353$）：

$$\sigma(2.0) = \frac{1}{1+e^{-2}} = \frac{1}{1+0.1353} = \frac{1}{1.1353} \approx \boxed{0.8808}$$

(2) Tanh（$e^2 \approx 7.389$，$e^{-2} \approx 0.1353$）：

$$\tanh(2.0) = \frac{7.389 - 0.1353}{7.389 + 0.1353} = \frac{7.2537}{7.5243} \approx \boxed{0.9640}$$

(3) ReLU：

$$\mathrm{ReLU}(2.0) = \max(0, 2.0) = \boxed{2.0}$$

$$\mathrm{ReLU}(-2.0) = \max(0, -2.0) = \boxed{0.0}$$

(4) Softmax（$\mathbf{z} = [3.0, 1.0, 0.5, -1.0]$）：

$e^{3.0} \approx 20.086$，$e^{1.0} \approx 2.718$，$e^{0.5} \approx 1.649$，$e^{-1.0} \approx 0.368$

总和 $S = 20.086 + 2.718 + 1.649 + 0.368 = 24.821$

$$\mathbf{p} = \left[\frac{20.086}{24.821}, \frac{2.718}{24.821}, \frac{1.649}{24.821}, \frac{0.368}{24.821}\right] \approx \boxed{[0.809,\ 0.109,\ 0.066,\ 0.015]}$$

验证：$0.809 + 0.109 + 0.066 + 0.015 = 0.999 \approx 1$（舍入误差）✓

(5) 数值稳定性证明：

设 $c = z_{\max}$，则：

$$\frac{e^{z_k - c}}{\sum_j e^{z_j - c}} = \frac{e^{z_k}/e^c}{\sum_j e^{z_j}/e^c} = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

分子分母同除以 e^c，结果不变。

对 $\mathbf{z} = [1000, 999, 998]$，$z_{\max} = 1000$，减去后得 $[0, -1, -2]$：

$$e^0 = 1,\ e^{-1} \approx 0.368,\ e^{-2} \approx 0.135; \quad S = 1.503$$

$$\mathbf{p} \approx \boxed{[0.665,\ 0.245,\ 0.090]}$$

(6) 值域总结：

激活函数	值域	约束
Sigmoid	$(0, 1)$	输出为正数，非零均值
Tanh	$(-1, 1)$	输出零均值，对称
ReLU	$[0, +\infty)$	非负，无上界
Softmax	$(0, 1)$ 且和为 1	构成概率分布

Q2-03 六种激活函数全面对比与选择指南

题目描述：

文档第 2.2 节介绍了阶跃函数、Sigmoid、Tanh、ReLU、Softmax 及其他常见激活函数。激活函数的选择直接影响网络的训练速度和性能，需要系统梳理。

问题：

(1) 填写下表，全面对比六种激活函数：

函数	公式	值域	梯度特性	典型使用位置
阶跃函数	$\mathbb{1}[x \geq 0]$	$\{0, 1\}$	_____	感知机（不用于 NN）
Sigmoid	$\dfrac{1}{1+e^{-x}}$	_____	最大值 0.25，饱和区趋 0	_____
Tanh	$\dfrac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$	_____	_____
ReLU	$\max(0, x)$	_____	_____	_____
Leaky ReLU	$\begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$	_____	_____	解决死亡 ReLU 问题
Softmax	$e^{z_k} / \sum e^{z_j}$	_____	_____	_____

(2) **”死亡 ReLU”问题**：在什么情况下 ReLU 神经元会”死亡”（永久输出 0）？Leaky ReLU 如何解决这个问题？两者在反向传播时有什么区别？

(3) **Tanh 与 Sigmoid 的关系**：可以证明 $\tanh(x) = 2\sigma(2x) - 1$。从这个关系出发，解释为什么 Tanh 通常比 Sigmoid 更适合隐藏层（从”零均值”角度分析）。

(4) **激活函数选择决策树**：根据以下场景，分别推荐最合适的激活函数并说明理由：

a. 多分类任务的输出层（10 个类别）；

b. 二分类任务的输出层；

c. 深层网络（50 层）的隐藏层；

d. 回归任务的输出层（预测连续值）；

e. 输出需要为零均值的隐藏层（如 RNN 中的隐藏状态）。

来源溯源：[文档第 2 章 2.2 激活函数 / 2.2.8 如何选择激活函数]

生活比喻：六种激活函数像六种”开关”

- **阶跃函数** = 硬开关：要么全开（1），要么全关（0），没有中间状态；切换瞬间导数无穷大，其余时刻导数为零——没法用梯度学习；
- **Sigmoid** = 调光旋钮（0 到 1）：从全暗到全亮平滑过渡，但旋到两端时旋钮很难转动（饱和区梯度极小）；
- **Tanh** = 双向调光旋钮（-1 到 1）：比 Sigmoid 对称，”中立态”在 0 而不是 0.5，更适合需要正负输出的场景；
- **ReLU** = 单向阀门：负压完全截断（输出 0），正压原样通过——简单、快，但一旦阀门被负压”卡死”就再也开不了（死亡 ReLU）；
- **Leaky ReLU** = 带小泄漏的单向阀：负压方向有一点点泄漏（αx），防止阀门被永久卡死；
- **Softmax** = 投票系统：把所有候选人的”得票数”（logits）转换成百分比（概率），所有人的占比加起来等于 100%。

参考答案与详细解析

(1) 六种激活函数对比表：

函数	公式	值域	梯度特性	使用位置
阶跃	$\mathbb{1}[x \geq 0]$	$\{0, 1\}$	不可微/恒为 0	感知机
Sigmoid	$1/(1+e^{-x})$	$(0, 1)$	最大 0.25，饱和趋 0	二分类输出层
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	$(-1, 1)$	最大 1，饱和趋 0	RNN 隐藏层
ReLU	$\max(0, x)$	$[0, +\infty)$	正区域恒 1，负区域恒 0	深层网络隐藏层
Leaky ReLU	见公式	$(-\infty, +\infty)$	正区域 1，负区域 α	替代 ReLU，防死亡
Softmax	$e^{z_k}/\sum e^{z_j}$	$(0, 1)$ 且和为 1	与输出层 CE 合并	多分类输出层

(2) 死亡 ReLU 与 Leaky ReLU：

当一个神经元的输入**永远为负**时（可能因为学习率过大导致权重大幅更新后落入负区域），ReLU 输出恒为 0，梯度恒为 0，该神经元**永远无法再通过梯度下降更新**——”死亡”了。

Leaky ReLU 在负值区域保留了一个小斜率 α（通常取 0.01），使负输入仍有非零输出和非零梯度（大小为 α），神经元不会被永久关闭。

反向传播区别：ReLU 在 $x \leq 0$ 处梯度为 0；Leaky ReLU 在 $x \leq 0$ 处梯度为 $\alpha > 0$，梯度信号仍可传递。

(3) Tanh 比 Sigmoid 更适合隐藏层：

Sigmoid 输出在 $(0,1)$ 之间，均值约为 0.5（非零均值）。当使用 Sigmoid 作为隐藏层时，下一层的输入均值偏正，导致权重梯度总是朝同一方向（正方向或负方向），更新效率低（" 之字形" 震荡）。Tanh 输出在 $(-1,1)$ 之间，**零均值对称**，下一层的输入均值接近 0，梯度方向更均衡，训练更稳定，收敛更快。

(4) 激活函数选择：

a. **多分类输出层** → **Softmax**：将 logits 转换为概率分布，配合交叉熵损失；

b. **二分类输出层** → **Sigmoid**：输出单一概率值 $p \in (0,1)$，配合二元交叉熵；

c. **深层网络隐藏层** → **ReLU**（或 Leaky ReLU）：计算高效，正区域梯度恒为 1，有效缓解梯度消失；

d. **回归输出层** → **无激活函数（线性输出）**：预测连续值不应限制范围，直接输出实数；

e. **需要零均值的隐藏层** → **Tanh**：输出范围 $(-1,1)$，零均值对称，适合 RNN 等需要正负输出的场景。

激活函数选择速查卡

- **隐藏层默认选** ReLU；若出现死亡神经元，改用 Leaky ReLU
- **多分类输出层**用 Softmax；**二分类输出层**用 Sigmoid
- **回归输出层**不加激活函数
- **RNN/需要零均值**用 Tanh

Q2-04 三层神经网络前向传播手动推导

题目描述：

文档第 2.3 节介绍了三层神经网络的结构和各层之间的信号传递。把数据在网络中的流动手算一遍，是理解前向传播的最直接方式。

网络结构：输入层（2 个节点）→ 隐藏层（3 个节点，ReLU）→ 输出层（2 个节点，Softmax）

给定权重和偏置（已初始化）：

$$W^{(1)} = \begin{pmatrix} 0.1 & 0.3 & -0.2 \\ 0.4 & -0.1 & 0.5 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0.1 \\ 0.2 \\ -0.1 \end{pmatrix}$$

$$W^{(2)} = \begin{pmatrix} 0.2 & -0.3 \\ 0.5 & 0.1 \\ -0.2 & 0.4 \end{pmatrix}, \quad b^{(2)} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$$

输入：$\mathbf{x} = \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix}$

问题：

(1) 写出各矩阵的形状，并验证矩阵乘法的维度匹配：

a. $\mathbf{x}$：_____，$W^{(1)}$：_____，$(W^{(1)})^T\mathbf{x}$（或 $\mathbf{x}^T W^{(1)}$）的形状：_____

b. $W^{(2)}$：_____，隐藏层输出：_____，最终输出：_____

(2) **第一层（隐藏层）前向传播**：计算线性变换 $\mathbf{z}^{(1)} = (W^{(1)})^T\mathbf{x} + b^{(1)}$（结果保留三位小数）。

(3) **ReLU 激活**：对 $\mathbf{z}^{(1)}$ 应用 ReLU，得到隐藏层输出 $\mathbf{h} = \mathrm{ReLU}(\mathbf{z}^{(1)})$。

(4) **第二层（输出层）前向传播**：计算 $\mathbf{z}^{(2)} = (W^{(2)})^T\mathbf{h} + b^{(2)}$（结果保留三位小数）。

(5) **Softmax 输出**：对 $\mathbf{z}^{(2)}$ 应用 Softmax，得到最终预测概率 $\hat{\mathbf{y}}$（结果保留三位小数）。模型预测该样本属于哪个类别？

来源溯源：[文档第 2 章 2.3.1 三层神经网络 / 2.3.2 各层之间的信号传递]

小白必读：前向传播的本质就是矩阵运算流水线

- **每一层做两件事**：(1) 线性变换：$\mathbf{z} = W^T\mathbf{x} + b$（加权求和）；(2) 非线性激活：$\mathbf{h} = f(\mathbf{z})$（激活函数）。
- **为什么用矩阵表示**：一层有多个神经元，每个神经元对所有输入都有一个权重。把所有权重打包成矩阵 W，一次矩阵乘法就完成了整层所有神经元的加权求和，既简洁又高效（GPU 专门优化矩阵运算）。
- **维度匹配是调试的关键**：前向传播出错 90% 是维度不匹配。养成”每步都检查形状”的习惯，是深度学习实践的基本功。

参考答案与详细解析

(1) 形状验证：

$\mathbf{x}$: $(2,1)$；$W^{(1)}$: $(2,3)$；$(W^{(1)})^T\mathbf{x}$: $(3,2)(2,1)=(3,1)$

$W^{(2)}$: $(3,2)$；隐藏层输出 $\mathbf{h}$: $(3,1)$；$(W^{(2)})^T\mathbf{h}$: $(2,3)(3,1)=(2,1)$

(2) 第一层线性变换 $\mathbf{z}^{(1)}=(W^{(1)})^T\mathbf{x}+b^{(1)}$：

$$(W^{(1)})^T=\begin{pmatrix}0.1 & 0.4\\ 0.3 & -0.1\\ -0.2 & 0.5\end{pmatrix}$$

$$(W^{(1)})^T\mathbf{x}=\begin{pmatrix}0.1\times1.0+0.4\times0.5\\ 0.3\times1.0+(-0.1)\times0.5\\ (-0.2)\times1.0+0.5\times0.5\end{pmatrix}=\begin{pmatrix}0.300\\ 0.250\\ 0.050\end{pmatrix}$$

$$\mathbf{z}^{(1)}=\begin{pmatrix}0.300\\ 0.250\\ 0.050\end{pmatrix}+\begin{pmatrix}0.1\\ 0.2\\ -0.1\end{pmatrix}=\boxed{\begin{pmatrix}0.400\\ 0.450\\ -0.050\end{pmatrix}}$$

(3) ReLU 激活：

$z_1^{(1)}=0.400>0$，$z_2^{(1)}=0.450>0$，$z_3^{(1)}=-0.050\le0$

$$\mathbf{h}=\mathrm{ReLU}(\mathbf{z}^{(1)})=\boxed{\begin{pmatrix}0.400\\ 0.450\\ 0.000\end{pmatrix}}$$

第三个神经元被 ReLU 截断为 0。

(4) 第二层线性变换 $\mathbf{z}^{(2)}=(W^{(2)})^T\mathbf{h}+b^{(2)}$：

$$(W^{(2)})^T=\begin{pmatrix}0.2 & 0.5 & -0.2\\ -0.3 & 0.1 & 0.4\end{pmatrix}$$

$$(W^{(2)})^T\mathbf{h}=\begin{pmatrix}0.2\times0.4+0.5\times0.45+(-0.2)\times0\\ -0.3\times0.4+0.1\times0.45+0.4\times0\end{pmatrix}=\begin{pmatrix}0.080+0.225+0\\ -0.120+0.045+0\end{pmatrix}=\begin{pmatrix}0.305\\ -0.075\end{pmatrix}$$

$$\mathbf{z}^{(2)}=\begin{pmatrix}0.305\\ -0.075\end{pmatrix}+\begin{pmatrix}0.1\\ -0.1\end{pmatrix}=\boxed{\begin{pmatrix}0.405\\ -0.175\end{pmatrix}}$$

(5) Softmax 输出：

$e^{0.405}\approx1.500$，$e^{-0.175}\approx0.840$，总和 $S=2.340$

$$\hat{\mathbf{y}}=\left[\frac{1.500}{2.340},\ \frac{0.840}{2.340}\right]\approx\boxed{[0.641,\ 0.359]}$$

类别 0 的概率为 64.1%，类别 1 的概率为 35.9%，模型预测该样本属于**类别 0**。

Q2-05 神经网络代码追踪：NumPy 实现

题目描述：

文档第 2.3.3 节给出了三层神经网络的代码实现。阅读并追踪代码是将数学与工程实现打通的关键一步。

```python
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def softmax(x):
    c = np.max(x)               # 数值稳定性处理
    exp_x = np.exp(x - c)
    return exp_x / np.sum(exp_x)

def relu(x):
    return np.maximum(0, x)

class ThreeLayerNet:
    def __init__(self, input_size, hidden1_size,
                 hidden2_size, output_size):
        self.params = {}
        # 使用 He 初始化（适合 ReLU）
        self.params['W1'] = np.random.randn(
            input_size, hidden1_size) * np.sqrt(2 / input_size)
        self.params['b1'] = np.zeros(hidden1_size)
        self.params['W2'] = np.random.randn(
            hidden1_size, hidden2_size) * np.sqrt(2 / hidden1_size)
        self.params['b2'] = np.zeros(hidden2_size)
        self.params['W3'] = np.random.randn(
            hidden2_size, output_size) * np.sqrt(2 / hidden2_size)
        self.params['b3'] = np.zeros(output_size)

    def predict(self, x):
        W1,b1 = self.params['W1'], self.params['b1']
        W2,b2 = self.params['W2'], self.params['b2']
        W3,b3 = self.params['W3'], self.params['b3']

        a1 = x  @ W1 + b1      # 行A
        z1 = relu(a1)           # 行B
        a2 = z1 @ W2 + b2      # 行C
        z2 = relu(a2)           # 行D
        a3 = z2 @ W3 + b3      # 行E
        y  = softmax(a3)        # 行F
        return y

net = ThreeLayerNet(784, 100, 50, 10)
x_sample = np.random.randn(1, 784)   # 单个样本
```

小白必读：矩阵乘法中"谁乘谁"容易搞混——记住这个规律

- **代码中的写法** `x @ W`：输入 x 在**左**，权重 W 在**右**。x 形状 (N, D_{in})，W 形状 $(D_{\text{in}}, D_{\text{out}})$，结果形状 (N, D_{out})。
- **与数学公式的对应**：数学上写 $\mathbf{z} = W^T\mathbf{x}$（列向量），代码中写 `z = x @ W`（行向量/批量）——两者等价，只是约定不同。
- **批量计算的优势**：`x @ W` 中 x 的每一行都是一个独立样本，矩阵乘法自动对所有样本并行计算——这正是深度学习能在 GPU 上高速运行的基础，GPU 矩阵乘法的吞吐量比逐样本循环高出数百倍。

参考答案与详细解析

(1) 形状追踪（`x_sample` 形状 `(1, 784)`）：

变量	行号	形状	计算说明
`a1`	A	`(1, 100)`	$[1, 784] \times [784, 100] + [100]$
`z1`	B	`(1, 100)`	ReLU 不改变形状
`a2`	C	`(1, 50)`	$[1, 100] \times [100, 50] + [50]$
`z2`	D	`(1, 50)`	ReLU 不改变形状
`a3`	E	`(1, 10)`	$[1, 50] \times [50, 10] + [10]$
`y`	F	`(1, 10)`	Softmax 不改变形状

(2) `c = np.max(x)` 的作用（数值稳定性）：

当 `x` 中有很大的值（如 1000），e^{1000} 超过 `float64` 的最大值（约 1.8×10^{308}），产生 `inf`，进而导致 `inf/inf = nan`，计算崩溃。

减去最大值后，最大项变为 $e^0 = 1$，其他项均 ≤ 1，不会溢出。由于证明 Softmax 结果不受常数偏移影响（Q2-02 第 (5) 题），此操作安全且必要。

(3) He 初始化 vs Xavier 初始化：

Xavier 初始化假设激活函数不截断信号（前向方差保持不变）。ReLU 将所有负值截断为 0，约有一半的神经元输出为 0，有效方差减半。He 初始化用 $\sqrt{2/n_{\text{in}}}$（比 Xavier 多因子 $\sqrt{2}$），恰好补偿 ReLU 使方差减半的效应，保持各层激活值的方差稳定，是 ReLU 网络的推荐初始化方式。

(4) 批量推理（`x_batch` 形状 `(32, 784)`）：

代码**无需任何修改**，NumPy 矩阵乘法自动支持批量：`a1 = x_batch @ W1 + b1`，形状为 `[32,784]@[784,100]+[100] = (32, 100)`。偏置 `b1` 形状为 `(100,)`，通过广播机制自动加到 32 个样本的每一行上。整个前向传播对 32 个样本并行计算，效率远高于逐样本循环。

(5) 参数总量计算：

层	权重参数	偏置参数
第 1 层 (784→100)	$784 \times 100 = 78400$	100
第 2 层 (100→50)	$100 \times 50 = 5000$	50
第 3 层 (50→10)	$50 \times 10 = 500$	10
合计	83900	160

$$\text{总参数量} = 83900 + 160 = \boxed{84060}$$

Q2-06 综合应用：手写数字识别全流程

题目描述：

文档第 2.4 节给出了手写数字识别（MNIST）的应用案例。MNIST 是深度学习的"Hello World"——28×28 像素的灰度图像，识别 0 到 9 共 10 个数字。

```python
import numpy as np

# 假设已加载 MNIST 测试集
# x_test: (10000, 784)  float32, 像素值归一化到 [0,1]
# t_test: (10000,)      int, 真实标签 (0-9)

def load_pretrained_weights():
    """返回预训练好的权重字典 (三层网络: 784->100->50->10) """
    pass  # 假设已实现

network = ThreeLayerNet(784, 100, 50, 10)
network.params = load_pretrained_weights()

# ===== 批量推理 =====
batch_size = 100
accuracy_count = 0

for i in range(0, len(x_test), batch_size):
    x_batch = x_test[i : i + batch_size]          # 行A
    y_batch = network.predict(x_batch)             # 行B
    p = np.argmax(y_batch, axis=1)                 # 行C
    accuracy_count += np.sum(p == t_test[i:i+batch_size])  # 行D

accuracy = accuracy_count / len(x_test)
print(f"测试集准确率: {accuracy:.4f}")

```

问题：

(1) **数据预处理**：代码注释中提到" 像素值归一化到 $[0,1]$"。原始 MNIST 像素值范围是 $[0,255]$，写出归一化公式，并解释为什么要归一化而不是直接使用原始像素值。

(2) **逐行分析**：

a. 行 A：`x_batch` 的形状是什么（第一次循环 $i=0$ 时）?

b. 行 B：`y_batch` 的形状是什么？每一行代表什么含义？

c. 行 C：`np.argmax(y_batch, axis=1)` 的作用是什么？`axis=1` 是什么意思？结果 `p` 的形状是什么？

d. 行 D：`p == t_test[i:i+batch_size]` 返回什么类型的数组？`np.sum` 对布尔数组求和会得到什么？

(3) **为什么用批量推理而不是逐图推理**：逐图处理（`batch_size=1`）功能上可以，但效率极低。解释批量推理在 CPU 和 GPU 上的效率优势。

(4) **输出解读**：设某张图片经过网络后的 Softmax 输出为：

小白必读：MNIST 数据集是什么？为什么从它开始？

- **MNIST 是什么**：由美国国家标准与技术研究院（NIST）收集的手写数字数据集，共 70000 张图片（60000 训练 + 10000 测试），每张 28×28 像素，灰度图，标签为 0 到 9 的整数。
- **为什么从 MNIST 开始**：数据小（每张图仅 784 个像素），类别少（10 类），任务清晰（分类数字），一个简单的三层网络就能达到 97% 以上的准确率——非常适合验证网络实现是否正确，是入门深度学习的" 标准测试题"。
- **数字"3" 被识别为什么样子**：网络把 $28 \times 28 = 784$ 个像素值（0 到 1 之间的灰度值）作为 784 维输入向量，经过三层变换，输出 10 个数字的概率分布，概率最大的那个类别就是预测结果。

参考答案与详细解析

(1) 数据归一化：

归一化公式：$x_{\text{norm}} = x_{\text{raw}}/255.0$，将 $[0, 255]$ 映射到 $[0, 1]$。

为什么归一化：

- 未归一化时，像素值量级为 $[0, 255]$，而权重通常初始化为小数（量级 10^{-2}），两者量级差距过大，加权求和后激活层输入极大，Sigmoid/Tanh 直接进入饱和区，梯度消失；ReLU 则产生巨大输出，可能引发梯度爆炸；
- 归一化后所有特征在同一量级范围，梯度下降更稳定，收敛更快。

(2) 逐行分析：

a. 行 A（$i = 0$）：`x_batch = x_test[0:100]`，形状 `(100, 784)`；

b. 行 B：`y_batch` 形状 `(100, 10)`，每行是一个样本的 10 个类别概率，各行加和为 1；

c. 行 C：`np.argmax(y_batch, axis=1)` 沿每行（`axis=1`）取最大值的索引，即每个样本的预测类别。`p` 形状为 `(100,)`，值域 $\{0, 1, \ldots, 9\}$；

d. 行 D：`p == t_test[...]` 返回布尔数组 `(100,)`，预测正确处为 `True`，错误处为 `False`。`np.sum` 将 `True` 计为 1、`False` 计为 0 求和，得到该 batch 的正确预测数量。

(3) 批量推理的效率优势：

- **CPU**：逐样本处理时，每次矩阵乘法规模小（$[1, 784] \times [784, 100]$），CPU 的 SIMD 向量化单元利用率低；批量后单次矩阵乘法规模大（$[100, 784] \times [784, 100]$），向量化利用率更高，同等时间内处理更多样本；
- **GPU**：GPU 有数千个并行计算核心，批量推理将 100 个样本合并为一次大矩阵运算，充分发挥并行性；逐样本时 GPU 大部分核心空闲，效率极低。

(4) Softmax 输出解读：

$\hat{\mathbf{y}} = [0.01, 0.02, 0.05, \mathbf{0.80}, 0.03, 0.02, 0.01, 0.03, 0.02, 0.01]$

a. 索引 3 对应的概率最大（0.80），模型预测为**数字"3"，置信度 80%**；

b. 数字"2"（索引 2）的概率为 0.05，即 5%——模型认为有 5% 的可能性是"2"；

c. 真实标签为"3"（索引 3），预测类别也是 3，**预测正确**。

(5) 更全面的评估指标：

推荐使用**混淆矩阵（Confusion Matrix）**及基于它的指标：

- **精确率（Precision）**：预测为某类的样本中，真正属于该类的比例——衡量”预测为正的可靠性”；
- **召回率（Recall）**：某类真实样本中，被正确预测出来的比例——衡量”漏检率”；
- **F1 分数**：精确率和召回率的调和平均 $F1 = 2PR/(P+R)$——综合两者的平衡指标。

对于类别不平衡问题（90% 是”0”），一个永远预测”0”的模型准确率 90%，但对其他 9 个类别的召回率均为 0，F1 分数极低，这能真实反映模型的无效性。

MNIST 三层网络的典型性能参考

- 随机猜测（10 类均匀）：准确率约 10%
- 简单线性分类器（无隐藏层）：约 92%
- 本章三层全连接网络（784-100-50-10）：约 97 ∼ 98%
- 卷积神经网络（LeNet-5）：约 99.3%
- 目前最优模型：$> 99.7\%$

现实应用场景

手写识别——从学术玩具到工业应用：
MNIST 诞生于 1998 年，至今仍是入门深度学习的标准数据集。但它的”后辈”们已经渗透日常生活：银行支票上的手写金额识别（识别引擎与本章网络原理相同，只是规模更大）、快递单上的手写地址解析、智能手表的手写输入……这些系统背后都是深度神经网络。从 MNIST 的 784 个像素到 ChatGPT 的万亿参数，**前向传播”层层加权求和 + 激活”的基本结构从未改变**——改变的只是深度、宽度和训练技巧。理解了本章，你就理解了这一切的起点。

2 本章小结：神经网络的”骨架”已经搭好

学完这 6 道题，整章内容可以浓缩成这几句话：

- **感知机 → 神经网络的关键一步**：把不可微的阶跃函数换成平滑的激活函数（Sigmoid/ReLU），梯度下降才能运作，深度学习才成为可能；
- **没有激活函数，多层 = 单层**：纯线性层叠加等价于一个线性变换，激活函数引入的非线性才是网络”表达力”的来源；
- **激活函数选择口诀**：隐藏层用 ReLU，多分类输出用 Softmax，二分类输出用 Sigmoid，回归输出不加激活；
- **前向传播的本质**：矩阵乘法 + 激活函数的流水线，每层做两件事：线性变换（$z = Wx + b$）+ 非线性激活（$h = f(z)$）；
- **批量计算的威力**：把 N 个样本打包成矩阵一次性算完，比逐样本循环快数百倍，这是现代深度学习能处理海量数据的基础；
- **MNIST = 深度学习的 Hello World**：像素归一化 → 三层网络前向传播 → Softmax 输出 → argmax 取预测类别，一条完整的推理链路。

给零基础读者的一句总比喻

如果把神经网络比作一台”模式翻译机”：

- **输入层** = 原材料仓库：接收原始数据（784 个像素值）；
- **隐藏层** = 加工车间：每一层提取更高级的特征——第一层可能学到”边缘”，第二层学到”笔画”，第三层学到”数字轮廓”；
- **激活函数** = 车间里的成型模具：没有它，每道工序只是简单拉伸，有了它，才能把原材料加工成各种形状；
- **输出层（Softmax）** = 质检台：把加工结果转化为”这是数字 0 的概率是 X%，数字 1 的概率是 Y%……”，取概率最大的就是预测答案；
- **前向传播** = 一件产品从原材料仓库流过所有车间到质检台的完整过程——本章讲的正是这条流水线怎么搭、怎么运转。